UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea Magistrale in Ingegneria Informatica

Elaborato in Network and Cloud Infrastructures

Slicing dinamico per flusso con soglia adattiva in reti SDN

Anno Accademico 2025/2026

Studenti
Castrese Basile DE9000181
Maria Coscione DE9000085
Valeria De Falco DE9000078

1 Introduzione

I meccanismi di qualità del servizio (QoS) operano su due piani: il data plane agisce su ogni pacchetto in ogni nodo, il control plane decide come allocare le risorse della rete.

Sul data plane, lo scheduling decide quale pacchetto trasmettere e quando. La disciplina più diffusa è la FIFO, che lascia il controllo della congestione agli host: le prestazioni di un flusso dipendono quindi dal traffico di tutti gli altri. Il problema emerge quando convivono TCP e UDP: il TCP rallenta quando rileva perdite, l'UDP no, ed è sempre più usato dalle applicazioni pesanti. Un solo flusso UDP può così lasciare quasi senza banda decine di flussi TCP, che si tratti di un attacco DoS o di un'applicazione legittima ma aggressiva.

Il Fair Queueing (FQ) e la sua variante pesata (WFQ) isolerebbero i flussi tra loro, ma richiedono una coda e uno stato per flusso in ogni nodo, e per questo sono poco diffusi. Il presente lavoro parte quindi dalla FIFO e la migliora senza sostituirla, ottenendo l'isolamento dal control plane, che in una rete SDN è centralizzato nel controller.

Il controller, basato su Ryu e OpenFlow 1.3, usa una forma di slicing dinamico: quando il collegamento verso un server è congestionato, i flussi che superano il doppio della loro parte equa vengono spostati temporaneamente su un percorso secondario a capacità ridotta (slice LOWER) e tornano su quello principale (slice UPPER) quando rientrano nella norma. Il progetto rispetta tre vincoli:

1. **nessun blocco automatico**: un flusso sospetto viene rallentato, non scartato, quindi un falso positivo costa banda, non la connessione;
2. **soglia adattiva**: la soglia segue il numero di flussi attivi, senza valori fissati a mano;
3. **governabilità**: un amministratore, o un'altra applicazione, può imporre le proprie decisioni, compreso il blocco, mentre la rete è in funzione.

2 Topologia di rete

La rete, emulata in Mininet, è composta da cinque switch Open vSwitch, gestiti da un'unica istanza di Ryu tramite OpenFlow 1.3, e da cinque host.

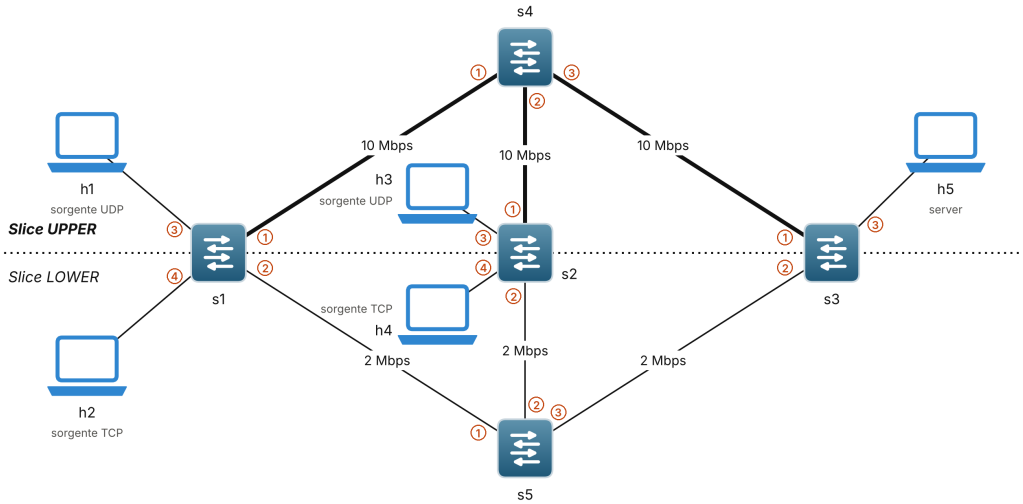


Figura 1: Topologia della rete

Gli switch s1, s2 ed s3 sono switch di accesso, ciascuno collegato ai due switch di transito s4 ed s5, che realizzano le due slice: la slice UPPER (s4) è il percorso principale, la slice LOWER (s5) quello secondario, a capacità ridotta. Le due sorgenti UDP aggressive (h1 e h3) sono collocate su switch diversi e ciascuna condivide il proprio switch con una sorgente TCP (h2 e h4): il traffico che arriva a s4 da uno stesso switch di accesso mescola quindi flussi aggressivi e flussi corretti, e bloccare la porta da cui entra colpirebbe anche questi ultimi. È quindi necessario intervenire sul singolo flusso. Tutto il traffico è diretto al server h5, collegato a s3, per cui il collo di bottiglia è il collegamento s4→s3. I parametri dei collegamenti sono riportati in Tabella 1.

Tabella 1: Parametri dei collegamenti.

Collegamento	**Banda**	**Ritardo**	**Coda**	**Porta**	**Ruolo**
host – switch di accesso	non limitata	–	–	3, 4	non deve essere il collo di bottiglia
s1, s2, s3 – s4 (slice UPPER)	10 Mbit/s	2 ms	100 pkt	1	percorso principale
s1, s2, s3 – s5 (slice LOWER)	2 Mbit/s	5 ms	100 pkt	2	percorso secondario, a capacità ridotta

La colonna *Porta* indica il numero di porta OpenFlow sullo switch di accesso.

Anelli. I due switch di transito creano anelli (per esempio s1→s4→s2→s5→s1), quindi una richiesta ARP inoltrata in broadcast circolerebbe all'infinito (broadcast storm). STP non è adatto, perché spegnerebbe due dei sei collegamenti tra switch, mentre il controller deve poter usare entrambe le slice. Si è scelto quindi un inoltro proattivo, con tutte le regole installate quando lo switch si collega e nessun flood, e un ARP proxy: gli switch di accesso inviano le richieste ARP al controller, che conosce IP e MAC di ogni host e risponde con un packet-out solo sulla porta dell'host che ha chiesto.

Negli esperimenti h5 (10.0.0.5) è il server vittima; h2 e h4 generano traffico TCP legittimo verso h5, mentre h1 e h3 inviano flussi UDP a rate costante.

3 Il controller

Il controller è diviso in cinque moduli. Il primo installa le regole quando uno switch si collega; gli altri quattro girano ciascuno in un proprio thread (`hub.spawn`) e comunicano solo attraverso dati condivisi: le misure del Monitor, la coda di ordini `orders` e il file `policy.json`.

Regole di inoltro. Le regole, installate quando lo switch si collega, derivano da una tabella interna con dpid, porte, IP e MAC di ogni host; per questo in Mininet porte e MAC sono assegnati a mano. Su s4 ed s5 basta una regola per destinazione. Sugli switch di accesso (s1, s2, s3), oltre all'invio dell'ARP al controller e alla consegna agli host locali, c'è una regola per ogni coppia mittente locale → destinatario remoto, con uscita sulla porta 1 (slice UPPER). Le priorità sono, dalla più alta: 100 per l'ARP, 30 per il blocco deciso dall'amministratore, 10 per l'inoltro, 1 per lo scarto di IPv6 e 0 per la table-miss.

Tabella 2: Regole OpenFlow installate dal controller, in ordine di priorità.

Prio	**Switch**	**Match**	**Azione**	**Scopo**
100	s1, s2, s3	`eth_type` = ARP	al controller	ARP proxy
30	s1, s2, s3 (Enforcement)	`eth_src`, `eth_dst` della coppia	scarta	blocco dell'amministratore
10	s1, s2, s3	`eth_src` locale, `eth_dst` remoto	porta 1 (UPPER) o 2 (LOWER)	inoltro per flusso
10	s1, s2, s3	`eth_dst` = host locale	porta dell'host	consegna locale
10	s4, s5	`eth_dst` = host	porta verso il suo switch di accesso	discesa verso l'host
1	tutti	`eth_type` = IPv6	scarta	IPv6 generato da Linux
0	tutti	qualsiasi pacchetto	al controller	table-miss

Le tre regole a priorità 10 non si sovrappongono: riguardano destinatari remoti, destinatari locali e switch diversi.

Monitoring. Una volta installate le regole, i pacchetti attraversano solo gli switch: per conoscere il traffico, il controller deve interrogarli. Ogni secondo il modulo richiede le statistiche delle regole (flow stats) agli switch di accesso e quelle delle porte (port stats) a s4. I contatori sono cumulativi, perciò il rate si ricava dalla differenza tra due letture consecutive:

$$r = \frac{8\,\Delta B}{\Delta t}$$

dove ΔB è l'aumento del contatore di byte nell'intervallo Δt e il fattore 8 converte i byte in bit. Il rate di ciascun flusso è misurato sullo switch di accesso del mittente (s1 per h1 e h2, s2 per h3 e h4), prima del collo di bottiglia s4→s3, e corrisponde quindi a quanto la sorgente trasmette; il carico del collegamento verso ogni switch di accesso si ricava invece dai byte trasmessi dalla porta corrispondente di s4.

Detection. Per ogni switch di accesso siano C la capacità del collegamento da s4 verso di esso (10 Mbit/s) e N il numero di flussi attivi (oltre 0,1 Mbit/s) che lo attraversano sulla slice UPPER negli ultimi 10 s; la parte equa di un flusso è C/N. Un flusso viene spostato sulla slice LOWER se, per 3 s consecutivi, il collegamento supera l'80% di C e il flusso supera il doppio della sua parte equa; torna sulla UPPER quando, per 5 s consecutivi, resta sotto la parte equa che avrebbe rientrando:

$$\text{spostamento: } r > 2\,\frac{C}{N} \qquad\qquad \text{riammissione: } r < \frac{C}{N+1}$$

Le due soglie sono distanti apposta: così un flusso vicino al limite non cambia slice ogni secondo. Il fattore 2, invece, serve a evitare falsi positivi.

Enforcement. Consuma la coda ed esegue gli ordini sullo switch d'ingresso del flusso. Spostamento e riammissione sono un `OFPFC_MODIFY_STRICT` della regola di coppia, con lo stesso match e la stessa priorità, che cambia solo l'uscita (porta 2 o 1): s4 ed s5 sanno già raggiungere ogni host dunque la loro regola non va modificata. La modifica non azzera i contatori, quindi il flusso resta misurato anche sulla slice LOWER, ed è

proprio questa misura a deciderne la riammissione. Il blocco è una regola senza azioni a priorità 30, che prevale su quella di coppia senza cancellarla; lo sblocco la rimuove con `OFPFC_DELETE_STRICT`.

Policy. Rilegge ogni secondo `policy.json`, scritto dall'amministratore o da un'altra applicazione, che contiene tre liste di coppie di IP: `block` (flussi da scartare), `lower` (flussi da tenere sulla slice LOWER senza riammissione automatica) e `allow` (flussi esclusi dai controlli, riportati sulla UPPER se erano stati spostati). I blocchi arrivano al modulo di Enforcement attraverso la stessa coda, le altre due liste al modulo di Detection. Se il file è illeggibile, resta valida la policy precedente.

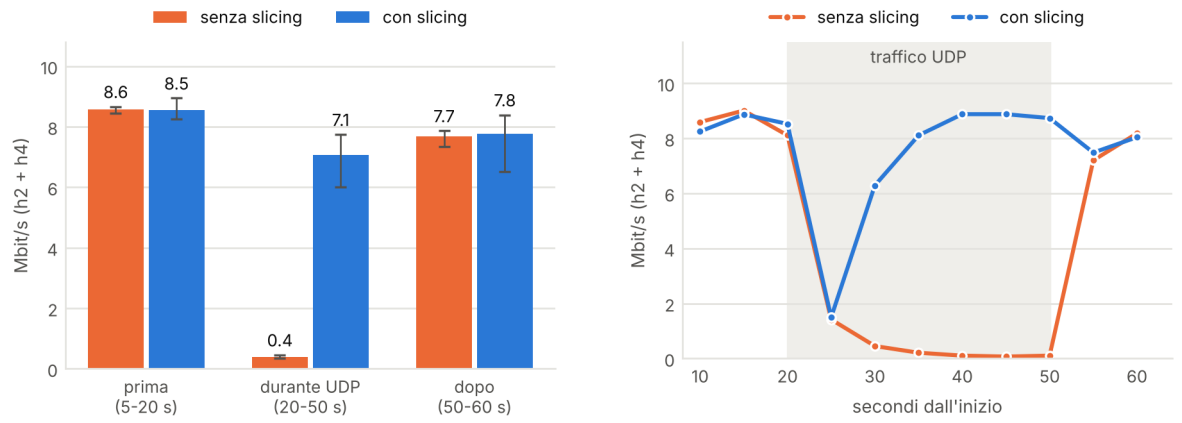
4 Sperimentazione

Scenario e metodo. Lo stesso script di traffico è lanciato in tre configurazioni: senza slicing dinamico (`DEFENSE_ENABLED = False`: il controller si limita a inoltrare e misurare); con slicing dinamico; con slicing dinamico e una decisione dell'amministratore, che prima della prova inserisce i due flussi UDP nella lista `allow` di `policy.json`, cioè li dichiara legittimi, e il modulo di Detection non li sposta. Da 0 s a 60 s h2 e h4 trasmettono in TCP verso h5; da 20 s a 50 s h1 e h3 inviano UDP a 6 Mbit/s ciascuno. Il valore è scelto a posta: ogni UDP da solo sta sotto la capacità del collegamento, ma con $N = 4$ supera la soglia $2C/N = 5$ Mbit/s, e insieme i due UDP offrono 12 Mbit/s al collo di bottiglia s4→s3. Ogni configurazione è ripetuta 3 volte (quella con allow una volta), riavviando Ryu e Mininet tra una prova e l'altra. Il throughput TCP è la somma di h2 e h4 nei report di iperf, uno ogni 5 s, escluso il primo (0–5 s), in cui iperf riporta un valore più alto del reale. Per l'UDP si usa il report del server h5, l'unico che sa quanti pacchetti sono arrivati e quanti sono andati persi.

Tabella 3: Risultati: media + (min-max) di 3 prove (allow: 1 prova), in Mbit/s;

Misura	**Senza slicing**	**Con slicing**	**UDP in allow**
TCP prima dell'UDP (5–20 s)	8,6 (8,5–8,7)	8,5 (8,2–8,9)	8,3
TCP durante l'UDP (20–50 s)	0,4 (0,3–0,5)	7,1 (6,0–7,7)	0,4
TCP a regime (30–50 s)	0,1 (0,1–0,2)	8,6 (8,5–9,0)	0,1
UDP ricevuto da h5, per flusso	4,3 (perdita 32%)	1,6 (perdita 74%)	4,2 (perdita 34%)

Risultati. Senza slicing dinamico si ripete il caso di studio: sulla coda FIFO l'UDP non rallenta e il TCP sì, e durante l'UDP il throughput TCP scende da 8,6 a 0,4 Mbit/s (Tabella 3 e Figura 2). Con lo slicing dinamico i due UDP vengono spostati sulla slice LOWER e il TCP risale a 7,1 Mbit/s in media sui 30 s e a 8,6 a regime, quanto prima dell'UDP. Gli UDP non vengono scartati: arrivano a h5 a 1,6 Mbit/s ciascuno, valore che comprende i primi secondi passati sulla UPPER; sulla LOWER i due flussi si dividono i 2 Mbit/s di s5. Prima dell'UDP le due configurazioni danno lo stesso throughput: il collegamento è già oltre l'80%, come resterà dopo lo spostamento, ma lo occupano i soli TCP e con $N = 2$ la soglia vale 10 Mbit/s; nessun TCP viene spostato. Con i flussi UDP in allow il TCP crolla come senza slicing (0,4 Mbit/s): il modulo Detection rispetta la decisione dell'amministratore.



(a) per fase

(b) ogni 5 s

Figura 2: Throughput TCP (h2 + h4), media di 3 prove: (a) per fase (prima, durante UDP, dopo); (b) nel tempo (in grigio i 30 s di traffico UDP).

Esecuzione Nella Figura 3a, pochi secondi dopo l'arrivo dell'UDP, con il collegamento al 91% e $N = 4$, h1 e h3 superano da 3 s la soglia di 5 Mbit/s e vengono spostati; nella Figura 3b, finito l'UDP, restano 5 s sotto $C/(N+1)$ e vengono riammessi. Lo spostamento è avvenuto 5 s dopo l'inizio dell'UDP in due prove su tre e dopo 10 s nella terza, in cui il collegamento è stato letto per tre secondi sotto l'80% e il conteggio è ripartito; la riammissione arriva sempre 6–7 s dopo la fine dell'UDP.

```
[MON] link s4 -> s3: 9.2 Mbit/s (91% della capacita')
[MON]    flusso 10.0.0.1 -> 10.0.0.5: 6.5 Mbit/s
[MON]    flusso 10.0.0.3 -> 10.0.0.5: 6.5 Mbit/s
[DET] 10.0.0.1 -> 10.0.0.5: 6.5 Mbit/s > soglia 5.0 (N=4) da 3 s -> SPOSTATO SU LOWER
[DET] 10.0.0.3 -> 10.0.0.5: 6.5 Mbit/s > soglia 5.0 (N=4) da 3 s -> SPOSTATO SU LOWER
[ENF] s1: 10.0.0.1 -> 10.0.0.5 ora esce dalla porta 2 (s5, slice LOWER)
[ENF] s2: 10.0.0.3 -> 10.0.0.5 ora esce dalla porta 2 (s5, slice LOWER)
```

(a) spostamento sulla slice LOWER

```
[DET] 10.0.0.1 -> 10.0.0.5: 0.0 Mbit/s < 3.3 da 5 s -> RIAMMESSO SU UPPER
[DET] 10.0.0.3 -> 10.0.0.5: 0.0 Mbit/s < 3.3 da 5 s -> RIAMMESSO SU UPPER
[ENF] s1: 10.0.0.1 -> 10.0.0.5 ora esce dalla porta 1 (s4, slice UPPER)
[ENF] s2: 10.0.0.3 -> 10.0.0.5 ora esce dalla porta 1 (s4, slice UPPER)
```

(b) riammissione sulla slice UPPER

Figura 3: Log di Ryu durante una prova con slicing dinamico.

Limiti. In una sola occasione, nella prova con allow, il controller ha spostato un flusso TCP: finito l'UDP, il conteggio dei flussi attivi era ancora alto e la soglia troppo bassa. Il flusso è stato riammesso pochi secondi dopo, senza perdite per il TCP complessivo: un errore costa un po' di banda, non la connessione.

Conclusioni Il lavoro mostra che l'isolamento tra flussi si può ottenere dal control plane, lasciando la FIFO sui collegamenti: durante il traffico UDP il throughput TCP passa da 0,4 a 7,1 Mbit/s, senza scartare alcun flusso in automatico, con una soglia che segue il numero di flussi attivi e con l'amministratore libero di intervenire. Restano margini di miglioramento: con pochi flussi la soglia diventa permissiva e una stima di N più accurata eviterebbe il falso positivo osservato; un possibile sviluppo è distinguere i flussi anche dal comportamento, perché un TCP rallenta quando perde pacchetti e un UDP no.